Încărcare fișier

- Terminal: **swipl nume_fisier.pl**
- Dacă ați modificat fișierul și vreți să se reîncarce conținutul: **make.**
- Ca să ieșiți din swipl: ctrl + d
- **Dacă ramane hanging probabil ca ați uitat . la final**

Erori

- **Singleton variables**
  - Regula are o variabila, dar a fost folosită o singura data
  - Exemple: f(X) :- ceva_ce_nu_contine_X. (nu-l folosesc pe X => nu imi pasa ce valoare are => pot sa îl scot: f(_) :- acelasi_ceva).
- **Arguments are not sufficiently instantiated**
  - Apare in general atunci cand se genereaza soluțiile si de exemplu vrem sa comparam valoarea unei variabile înainte sa o initializam
  - De cele mai multe ori ajuta sa schimbăm ordinea scopurilor pentru a ne asigura ca se instantiaza variabilele înainte sa le folosim

Debug

- Cel mai util: **trace** pentru a vedea care sunt scopurile care se satisfac si cu ce legari
- Am făcut următorul predicat ca exemplu:

```
%% all_numbers_until_0/2
%% all_numbers_until_0(?N, ?L)
all_numbers_until_0(0, [0]).
all_numbers_until_0(N, [N | L]) :- N > 0, N1 is N - 1, all_numbers_until_0(N1, L).
```

- Funcția primește un număr N si o lista L care contine toate numerele de la N la 0
- **Exemple de scopuri:**
  - all_numbers_until_0(0, [0]).
    - Este all_numbers_until_0(0, [0]) adevarat? yes
  - all_numbers_until_0(0, [1]).
    - Este all_numbers_until_0(0, [1]) adevarat? nope
  - all_numbers_until_0(N, [3, 2, 1, 0]).
    - Care este valoarea lui N pentru care L = [3, 2, 1, 0]?
    - Afișează N = 3 si după dacă îi dați ; încearca sa resatisfaca scopul si întoarce false pentru ca nu exista alta valoare in afara de 3 pentru care all_numbers_until_0(N, [3, 2, 1, 0]) este true)

- all_numbers_until_0(5, L).
  - Care este lista L pentru care all_numbers_until_0(5, L) este adevarat?
  - Se va genera lista L pe baza regulilor definite
  - Pentru ca all_numbers_until_0(5, L) din regula 2 rezulta ca L = [5 | L1] si all_numbers_until(4, L1). La fel se face si pentru L1 si tot asa pana ce se descrie complet L.
  - Se obtine prima solutie L = [5, 4, 3, 2, 1, 0]
  - Mai departe dacă îi dați ; încearca sa resatisfaca scopul si întoarce false
- Pentru activarea tracing-ului, scrieti **trace. in terminal, nu in fisier**
- După asta o sa vedeți ca apare un **[trace]** in fata si orice scop îi veți da se vor afisa apelurile care se fac in spate si ce legari se obțin

```
1 ?- trace.
true.

[trace] 1 ?- all_numbers_until_0(2, L).
   Call: (10) all_numbers_until_0(2, _8294) ? creep
   Call: (11) 2>0 ? creep
   Exit: (11) 2>0 ? creep
   Call: (11) _11004 is 2+ -1 ? creep
   Exit: (11) 1 is 2+ -1 ? creep
   Call: (11) all_numbers_until_0(1, _9488) ? creep
   Call: (12) 1>0 ? creep
   Exit: (12) 1>0 ? creep
   Call: (12) _14788 is 1+ -1 ? creep
   Exit: (12) 0 is 1+ -1 ? creep
   Call: (12) all_numbers_until_0(0, _13272) ? creep
   Exit: (12) all_numbers_until_0(0, [0]) ? creep
   Exit: (11) all_numbers_until_0(1, [1, 0]) ? creep
   Exit: (10) all_numbers_until_0(2, [2, 1, 0]) ? creep
L = [2, 1, 0] ;
   Redo: (12) all_numbers_until_0(0, _13272) ? creep
   Call: (13) 0>0 ? creep
   Fail: (13) 0>0 ? creep
   Fail: (12) all_numbers_until_0(0, _13272) ? creep
   Fail: (11) all_numbers_until_0(1, _9488) ? creep
   Fail: (10) all_numbers_until_0(2, _8294) ? creep
false.
```

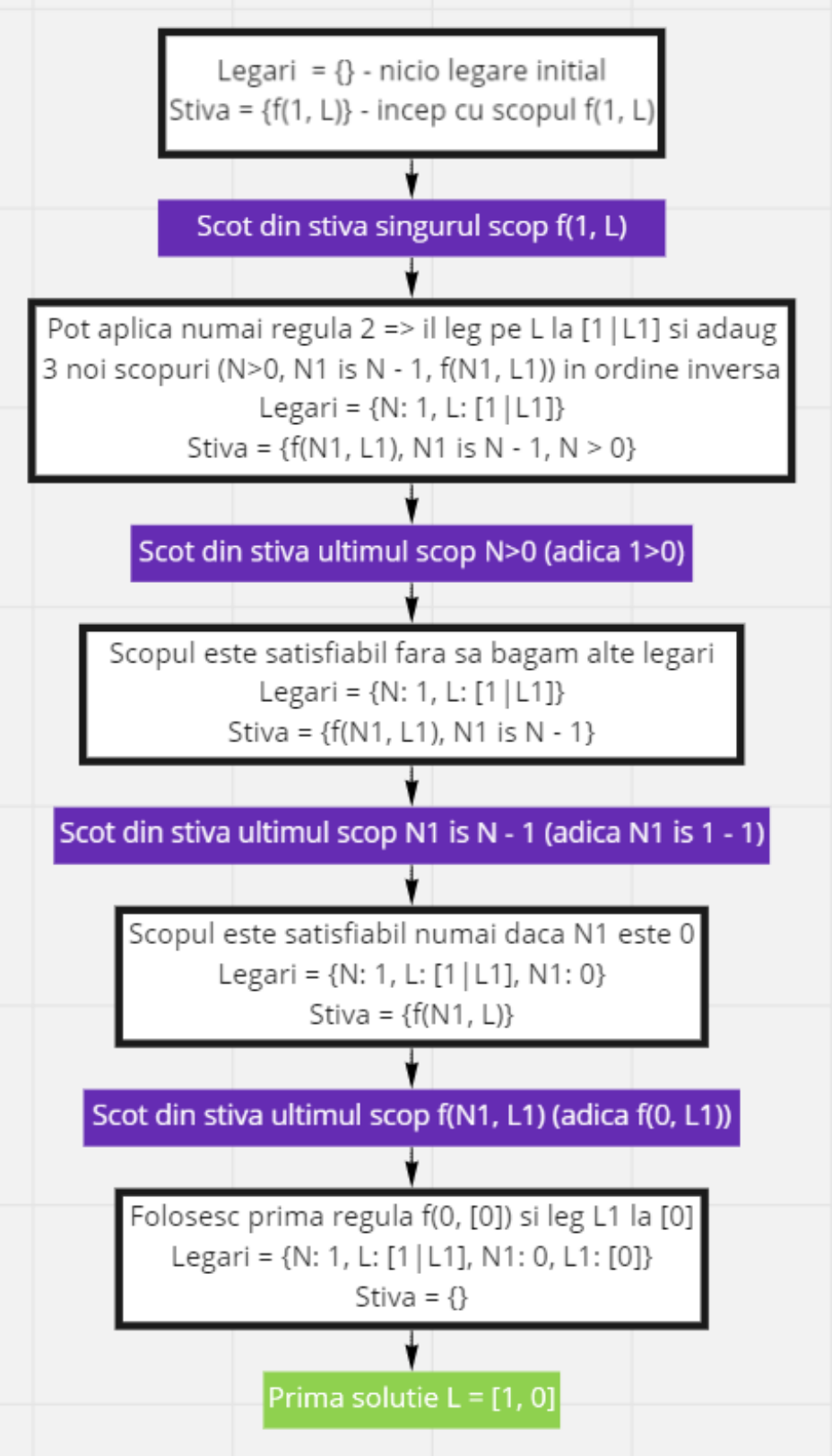
Ce înseamnă chestia asta?

- **Call: se încearcă satisfacerea unui scop:**
  - Cu variabile
    - ex: all_numbers_until_0(2, _8294)
    - _8294 e ce incerc sa aflu
    - Care sunt valorile lui _8294 pentru care all_numbers_until_0(2, _8294) este true?
  - Fara variabile
    - ex: 2 > 0?
- **Exit: s-a găsit o soluție pentru un scop**
  - dacă avea variabile => am găsit o instanțiere a lor pentru care scopul e satisfăcut
  - dacă nu avea variabile => e o condiție simpla care se respecta
- **Fail: nu pot sa satisfac scopul**
  - Cu variabile
    - all_numbers_until_0(0, [1 | X]) nu este satisfiabil indiferent ce valoare are X si o sa dea fail
  - Fara variabile
    - 0 > 0? nope
- **De unde își da seama de ce call-uri sa faca?**
  - Din reguli
  - Pentru all_numbers_until_0 am următoarele 2 reguli (am folosit f ca lene):
    f(0, [0]).
    f(N, [N | L]) :- N > 0, N1 is N - 1, f(N1, L).

Exemplu scopul f(1, L)

```
[trace] 5 ?- all_numbers_until_0(1, L).
   Call: (10) all_numbers_until_0(1, _216) ? creep
   Call: (11) 1>0 ? creep
   Exit: (11) 1>0 ? creep
   Call: (11) _2924 is 1+ -1 ? creep
   Exit: (11) 0 is 1+ -1 ? creep
   Call: (11) all_numbers_until_0(0, _1408) ? creep
   Exit: (11) all_numbers_until_0(0, [0]) ? creep
   Exit: (10) all_numbers_until_0(1, [1, 0]) ? creep
L = [1, 0] .
```

Iar asta e logica din spate



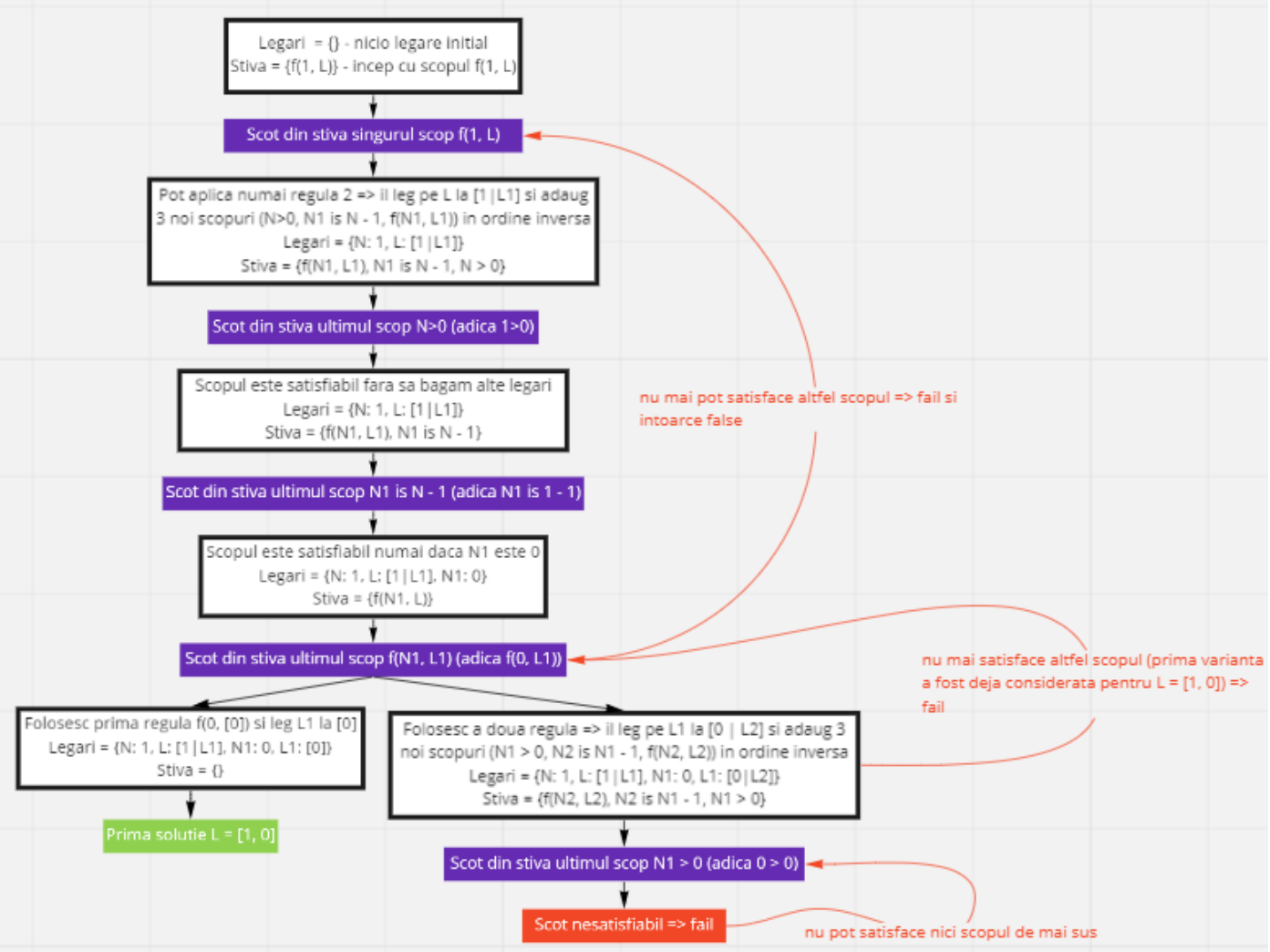
Resatisfacerea scopului

Inainte am dat . pentru ca ne doream numai prima valoare a lui L. Dacă exista mai multe putem sa incercam resatisfacerea unui scop cu ;

```
[trace] 5 ?- all_numbers_until_0(1, L).
   Call: (10) all_numbers_until_0(1, _1560) ? creep
   Call: (11) 1>0 ? creep
   Exit: (11) 1>0 ? creep
   Call: (11) _4268 is 1+ -1 ? creep
   Exit: (11) 0 is 1+ -1 ? creep
   Call: (11) all_numbers_until_0(0, _2752) ? creep
   Exit: (11) all_numbers_until_0(0, [0]) ? creep
   Exit: (10) all_numbers_until_0(1, [1, 0]) ? creep
L = [1, 0] ;
   Redo: (11) all_numbers_until_0(0, _2752) ? creep
   Call: (12) 0>0 ? creep
   Fail: (12) 0>0 ? creep
   Fail: (11) all_numbers_until_0(0, _2752) ? creep
   Fail: (10) all_numbers_until_0(1, _1560) ? creep
false.
```

Se mai întoarce si false pentru ca nu exista alta valoare a lui L pentru care f(1, L) sa fie true.

De ce? Se reiau toate scopurile in ordine inversa (backtracking) si se cauta noi legari.



Ce fac dacă nu vreau decat prima soluție? Operatorul cut !

Apelez all_numbers_until_0(1, L), !. in loc de all_numbers_until_0(1, L).